Module 5 Software Assurance Report

Jonah Thomas
Modern Software Concepts Using Python
Module 5: Software Assurance and Security

1. Project Overview

This project hardens the GradCafe Flask and PostgreSQL application developed in earlier modules. The application collects and loads GradCafe applicant records, stores them in PostgreSQL, performs analytical queries, and displays the results through a Flask webpage. Module 5 adds software-assurance practices including static analysis, SQL injection defenses, least-privilege database access, dependency analysis, reproducible installation, supply-chain scanning, and continuous integration.

The final application passes 23 automated tests with 100 percent source-code coverage. Pylint reports a score of 10.00 out of 10 with no remaining errors or warnings. The dependency scan found no vulnerable dependency paths, and the final source-only Snyk Code scan reported zero issues.

2. Reproducible Installation and Packaging

The project includes both `requirements.txt` and `setup.py`. The requirements file lists the application's runtime dependencies along with the testing, documentation, linting, and dependency-analysis tools required for the assignment. These include Flask, Beautiful Soup, lxml, psycopg, python-dotenv, Pytest, pytest-cov, Pylint, pydeps, Sphinx, and the Read the Docs theme.

The `setup.py` file makes the project installable as a Python package. Packaging improves import consistency between local execution, testing, and continuous integration. It also supports editable installation with `pip install -e .`, which allows source-code changes to be used immediately without reinstalling the package. This reduces path-related problems and helps prevent "works on my machine" behavior.

Installation with pip and venv

A fresh pip environment can be created with:

```
py -3.12 -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
python -m pip check
```

The installation was tested in a newly created environment. The application package imported successfully, `pip check` reported no broken requirements, Pylint returned 10.00 out of 10, and all tests passed with 100 percent coverage.

Installation with uv

The alternate uv installation path is:

```
uv venv .venv --python 3.12
uv pip install --python .\.venv\Scripts\python.exe -r requirements.txt
uv pip install --python .\.venv\Scripts\python.exe -e .
uv pip check --python .\.venv\Scripts\python.exe
```

The uv installation was also tested in a clean environment. The project imported successfully and produced the same passing test and lint results as the pip installation.

3. SQL Injection Defenses

The SQL layer was refactored to prevent SQL injection and unsafe query construction. User-controlled values are never inserted directly into SQL text through f-strings, string concatenation, or `.format()`. Instead, user values are passed separately to `cursor.execute()` through `%s` placeholders and parameter lists.

Dynamic SQL components such as table or column names are constructed with `psycopg.sql.SQL` and `sql.Identifier`. `sql.Identifier` safely quotes identifier values so malicious text cannot be interpreted as executable SQL syntax. The code also separates statement construction from statement execution. A composed SQL statement is created first, while values are supplied separately when the cursor executes the statement.

Each analysis query contains an inherent `LIMIT`. A validation helper converts the requested limit to an integer and clamps it to an allowed range from 1 through 100. Invalid, missing, negative, zero, or oversized values are replaced or constrained to a safe value. This prevents a user from requesting an unbounded result set or supplying malicious text as a limit.

Automated tests exercise invalid and out-of-range values. The application safely handles the inputs without exposing all database records, executing injected SQL, or crashing the analysis endpoint.

4. Least-Privilege Database Configuration

Database credentials are no longer hard-coded in the source code. The application reads the following environment variables:

DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASSWORD

A `.env.example` file is included with placeholder values. The real `.env` file is excluded through `.gitignore`, so passwords and local connection details are not committed to GitHub.

A PostgreSQL role named `gradcafe_app` was created for normal application use. This role is not a superuser, cannot create databases, cannot create roles, does not own the applicants table, and cannot create objects in the public schema. It was granted only `SELECT` and `INSERT` access because the application must read analysis data and insert newly scraped records.

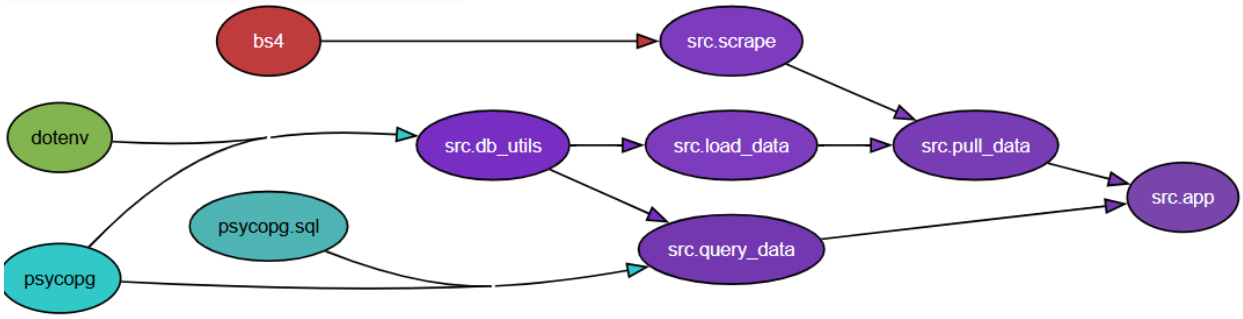
The relevant permissions were configured with SQL similar to:

```
GRANT SELECT, INSERT
ON TABLE applicants
TO gradcafe_app;

REVOKE UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER
ON TABLE applicants
FROM gradcafe_app;

REVOKE CREATE
ON SCHEMA public
FROM gradcafe_app;
```

This configuration was verified by successfully executing read and insert operations through the application account. An attempted `DROP TABLE applicants` operation failed with an insufficient-privilege error. Destructive database setup and teardown during testing use a separate administrative connection through `TEST_DATABASE_URL`, while normal application execution continues to use the restricted account.

5. Dependency Analysis

The Python dependency graph was generated with pydeps and Graphviz and saved as `dependency.svg`.



The graph shows that `src.app` is the Flask entry point and depends on both `src.pull_data` and `src.query_data`. The `src.pull_data` module coordinates scraping and database insertion by using `src.scrape` and `src.load_data`. Both `src.load_data` and `src.query_data` depend on `src.db_utils`, which centralizes environment-based PostgreSQL connection handling. The `src.query_data` module also depends directly on `psycopg.sql` for safe SQL composition. The scraper uses Beautiful Soup to parse GradCafe HTML records. The graph demonstrates that database configuration is centralized rather than duplicated across modules. It also shows a clear separation between web presentation, data collection, persistence, and analysis responsibilities.

The graph can be regenerated with:

```
python -m pydeps .\src `
   --noshow `
   -T svg `
   -o dependency.svg `
   --exclude flask flask.* `
   --max-bacon 2 `
   --rankdir LR
```

6. Static Analysis and Automated Testing

Pylint was run only against the Python files in `src`, as required:

```
python -m pylint .\src --fail-under=10
```

The final output was:

Your code has been rated at 10.00/10

The source code contains no remaining Pylint warnings or errors.

```
(.venv) PS C:\Users\jctho\Desktop\jhu_software_concepts\module_5> python -m pytest
.......................                                                  [100%]
=================================== tests coverage ===================================
_____________________ coverage: platform win32, python 3.12.10-final-0 _____________________

Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
src\__init__.py         0      0   100%
src\app.py             39      0   100%
src\db_utils.py        25      0   100%
src\load_data.py       86      0   100%
src\pull_data.py       12      0   100%
src\query_data.py      42      0   100%
-------------------------------------------------
TOTAL                 204      0   100%
Required test coverage of 100% reached. Total coverage: 100.00%
23 passed in 4.18s
(.venv) PS C:\Users\jctho\Desktop\jhu_software_concepts\module_5> python -m pylint .\src

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

The Pytest suite contains 23 tests covering the Flask page, button behavior, analysis formatting, database insertion, duplicate handling, helper functions, limit validation, and end-to-end application behavior. The project's `pytest.ini` requires 100 percent source-code coverage. The final local execution produced 23 passing tests and 100.00 percent coverage.

Supporting text evidence is also included in:

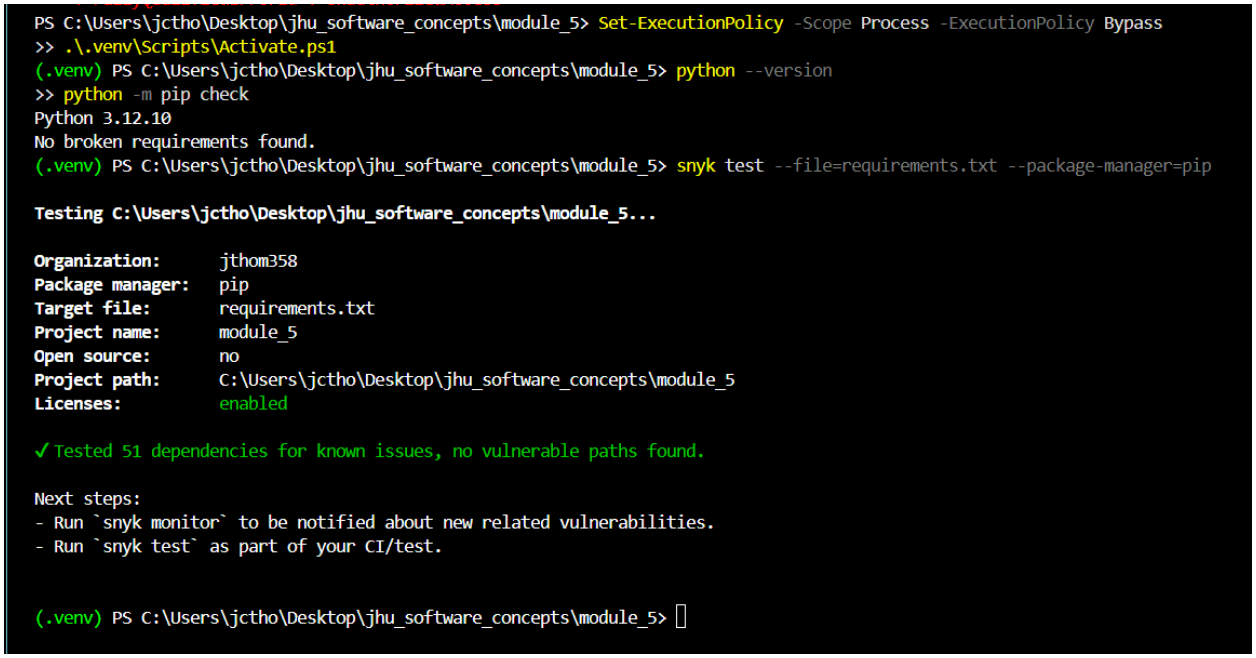
pylint_summary.txt
coverage_summary.txt

7. Snyk Dependency Scan

The required Snyk open-source dependency scan was run with:

snyk test --file=requirements.txt --package-manager=pip

The scan tested 51 dependencies and reported no vulnerable dependency paths. Therefore, no dependency removals or version patches were required at the time of submission.



8. Snyk Code Static Analysis — Extra Credit

The optional Snyk Code scan was run against the project source with:

snyk code test src

An initial scan of the entire `module_5` directory included the local `.venv` directory and reported numerous findings inside third-party libraries such as pip, Pytest, Sphinx, Flask, and Werkzeug. These files were installed dependencies and were not authored as part of the project. The scan was therefore limited to the submitted source directory so that the results represented the application's own code.

The initial source review identified that the Flask application was started with debug mode enabled. This was remediated by disabling debug mode in `src/app.py`. After the change, the final source-only Snyk Code scan completed with zero issues.

```
(.venv) PS C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src> snyk code test
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Uploading
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
- Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
\ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
| Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving
/ Snyk Code analysis for C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src - Retrieving

Testing C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src ...


 Test Summary

   Organization:      jthom358
   Test type:         Static code analysis
   Project path:      C:\Users\jctho\Desktop\jhu_software_concepts\module_5\src

   Total issues:   0
```

9. Continuous Integration

A GitHub Actions workflow was added at:

.github/workflows/module5-ci.yml

A matching copy is included inside the submission folder at:

module_5/.github/workflows/ci.yml

The workflow runs on pushes and pull requests that affect Module 5. It contains four separate jobs:

1. Pylint with `--fail-under=10`
2. Dependency graph generation and validation
3. Snyk dependency scanning
4. PostgreSQL-backed Pytest execution with 100 percent coverage enforcement

The dependency-graph job installs Graphviz, regenerates `dependency.svg`, verifies that the file exists and is nonempty, and uploads it as an artifact. The Pytest job starts PostgreSQL as a GitHub Actions service container and supplies a dedicated test database connection through environment variables. The Snyk job reads its authentication credential from the encrypted GitHub Actions secret named `SNYK_TOKEN`; the token is not stored in the repository.

The completed workflow passed all four jobs.

10. Conclusion

Module 5 converted the GradCafe application into a more secure and reproducible software project. SQL values are parameterized, dynamic identifiers are safely composed, and every analysis query has a validated limit. Database access follows the principle of least privilege, while administrative access is isolated to test setup and schema management. The project installs successfully through both pip and uv, passes all tests with full coverage, achieves a perfect Pylint score, and produces a validated dependency graph. Snyk found no vulnerable dependency paths, and the final source-only static analysis reported zero issues. GitHub Actions now enforces these quality and security checks automatically on every relevant push and pull request.